

Teste de Unidade Estrutural

Prof. Iaçanã Ianiski Weber



Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2nd edition. 2017.

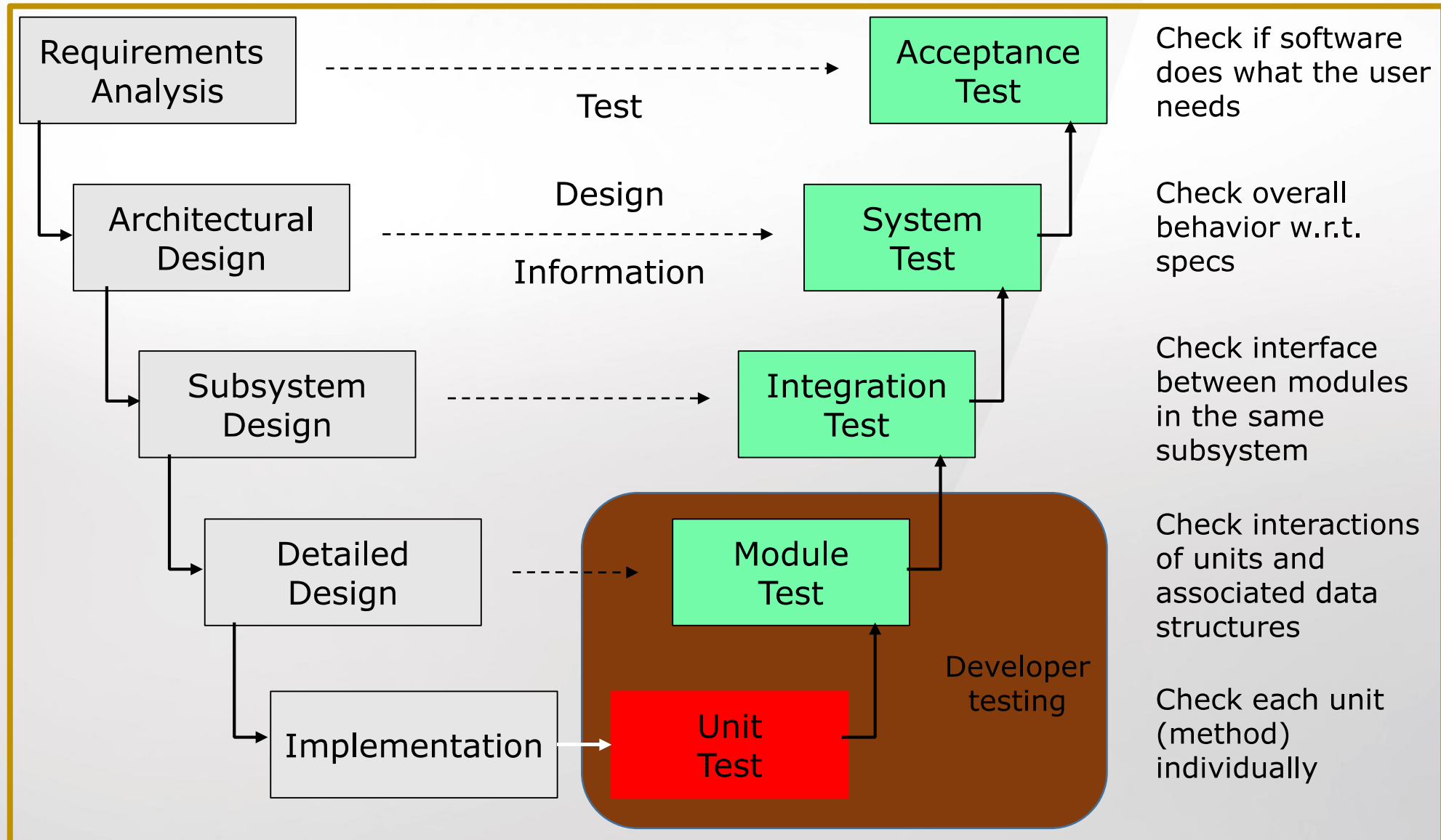
✓ <https://cs.gmu.edu/~offutt/softwaretest/>

- Seções 7.3 e 8.3

Delamaro, M.E.; Maldonado J.C.; Jino, M. ***Introdução ao Teste de Software***. Elsevier. 2007.

✓ <https://www.amazon.com.br/Introdu%C3%A7%C3%A3o-Software-M%C3%A1rcio-Eduardo-Delamaro/dp/8535226346>

- Capítulo 4



- ✓ Técnicas:
 - Funcional (Capítulo 2)
 - Estrutural (Capítulo 4)
 - Baseada em Erros
- ✓ A diferença está na fonte utilizada para *definir os requisitos de teste!!*
- ✓ Mas o **objetivo** das técnicas é similar:
 - Conduzir o processo de teste de uma forma mais sistemática
 - Escolher casos de teste que representem o domínio

- A técnica estrutural (ou caixa branca) estabelece os requisitos de teste **com base em uma dada implementação**, requerendo a execução de partes ou de componentes elementares do programa
- Os **caminhos lógicos** do software são testados, fornecendo-se casos de teste que põem à prova tanto conjuntos específicos de condições e/ou laços
- Os critérios são classificados com base na complexidade, no fluxo de controle e no fluxo de dados, e serão discutidos mais detalhadamente no decorrer desta aula

A técnica estrutural apresenta uma série de **limitações** e **desvantagens** decorrentes das limitações inerentes à atividade de teste de programa como estratégia de validação

- não existe um procedimento de teste de propósito geral que possa ser usado para provar a correção de um programa
- dados dois programas, é indecidível se eles computam a mesma função
- é indecidível, em geral, se dois caminhos de um programa, ou de programas diferentes, computam a mesma função
- é indecidível, em geral, se um dado caminho é executável, ou seja, se existe um conjunto de dados de entrada que leve à execução desse caminho

É importante ressaltar, ainda, as **limitações** inerentes à técnica estrutural:

- caminhos ausentes: se o programa não implementa algumas funções, não existirá um caminho que corresponda àquela função; conseqüentemente, nenhum dado de teste será requerido para exercitá-la
- correção coincidente: o programa pode apresentar, coincidentemente, um resultado correto para um dado em particular de entrada, satisfazendo um requisito de teste e não revelando a presença de um defeito; entretanto, se escolhido outro dado de entrada, o resultado obtido poderia ser incorreto

Independentemente dessas desvantagens, a técnica estrutural é vista como **complementar às demais técnicas** de teste existentes, uma vez que cobre *classes distintas de defeitos!*

As informações obtidas pela aplicação de critérios estruturais têm sido consideradas relevantes para as atividades de manutenção, depuração e avaliação da confiabilidade (*reliability*) de software

Conceito:

- ✓ É um método de projeto de testes que usa a *estrutura de controle* do projeto procedimental para derivar casos de teste
- ✓ Conhecido também como *Teste de Caixa Branca*
- ✓ Baseia-se num minucioso exame dos detalhes procedimentais
- ✓ Caminhos lógicos do software são testados
- ✓ Não é viável testar todos os caminhos lógicos de um programa (isto é, *teste exaustivo*)

- ✓ Técnica de Teste de Caixa Branca
- ✓ Calcula-se a complexidade lógica do software e utiliza esta medida como base para descobrir os caminhos básicos do software, isto é, o *conjunto básico de caminhos de execução*
- ✓ Os casos de teste derivados para executar os caminhos básicos tem *a garantia de executar cada instrução pelo menos uma vez*
- ✓ A maioria dos critérios da técnica estrutural utiliza uma representação de programa conhecida como “Grafo de Fluxo de Controle” (**GFC**) ou “Grafo de Programa”

- ✓ No GFC um programa P pode ser decomposto em um conjunto de blocos disjuntos de comandos
- ✓ A execução do primeiro comando de um bloco acarreta a execução de todos os outros comandos desse bloco
- ✓ Todos os comandos de um bloco, possivelmente com exceção do primeiro, têm um único predecessor e exatamente um único sucessor, exceto possivelmente o último comando

- ✓ Usualmente, a representação de um programa P como um GFC, G :

$$G = (N, E, s)$$
- ✓ Consiste em estabelecer uma correspondência entre vértices (nós) e blocos e em indicar possíveis fluxos de controle entre blocos por meio das arestas (arcos)
- ✓ Assume-se que um GFC é um grafo orientado, com um único nó de entrada $s \in N$ e um único nó de saída $o \in N$, no qual cada vértice representa um bloco indivisível de comandos e cada aresta representa um possível desvio de um bloco para outro
- ✓ A partir do GFC podem ser escolhidos os elementos que devem ser executados, caracterizando assim o teste estrutural

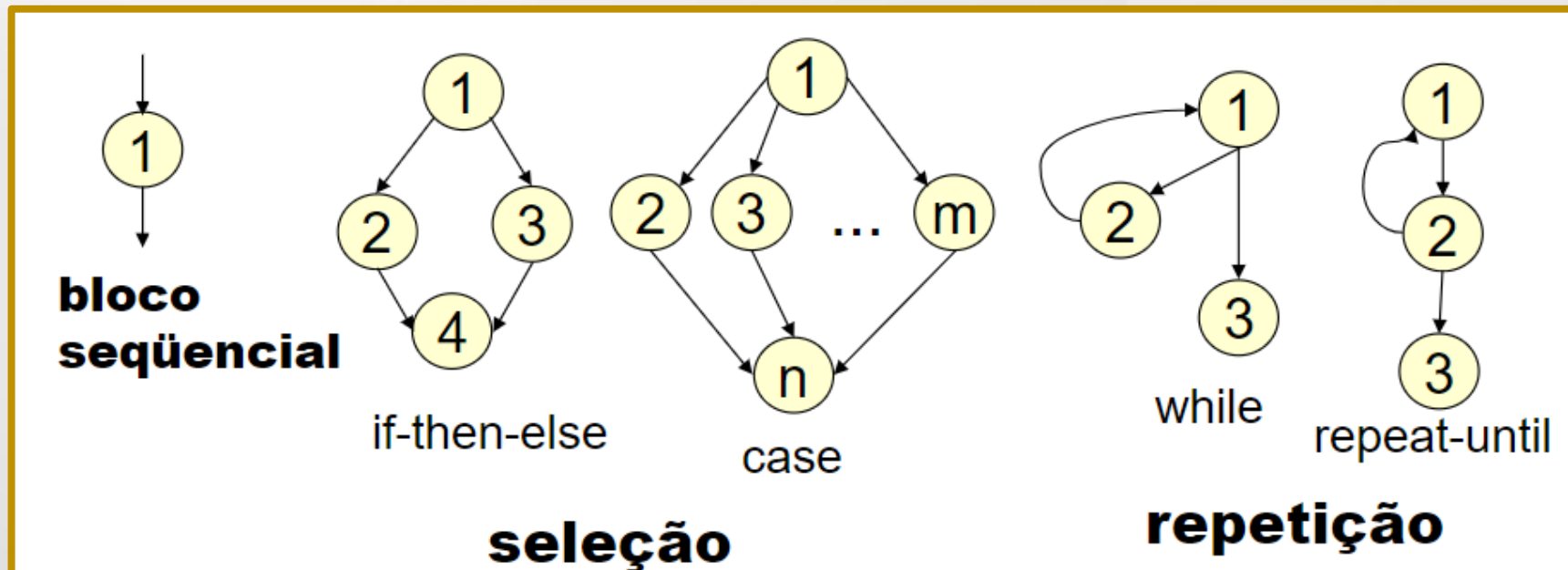
- ✓ Seja um GFC $G = (N, E, s)$ em que N representa o conjunto de nós, E o conjunto de arcos, e s o nó de entrada. Um “caminho” é uma sequência finita de nós $(n_1, n_2, \dots, n_k)$, $k \geq 2$, tal que existe um arco de n_i para n_{i+1} para $i = 1, 2, \dots, k - 1$.
- ✓ Um caminho é um “caminho simples” se todos os nós que compõem esse caminho, exceto possivelmente o primeiro e o último, são distintos; se todos os nós são distintos, diz-se que esse caminho é um “caminho livre de laço”.
- ✓ Um “caminho completo” é aquele em que o primeiro nó é o nó de entrada e o último nó é um nó de saída do grafo G .
- ✓ Seja $IN(x)$ e $OUT(x)$ o número de arcos que entram e que saem do nó x , respectivamente. Assumimos $IN(s) = 0$, tal que s é o nó de entrada, e $OUT(o) = 0$, tal que o é o nó de saída.

- ✓ Seja um GFC $G = (N, E, s)$ em que N representa o conjunto de nós, E o conjunto de arcos, e s o nó de entrada. Um “caminho” é uma sequência finita de nós $(n_1, n_2, \dots, n_k)$, $k \geq 2$, tal que existe um arco de n_i para n_{i+1} para $i = 1, 2, \dots, k - 1$.
- ✓ Um caminho é um “caminho simples” se todos os nós que compõem esse caminho, exceto possivelmente o primeiro e o último, são distintos; se todos os nós são distintos, diz-se que esse caminho é um “caminho livre de laço”.
- ✓ Um “caminho completo” é aquele em que o primeiro nó é o nó de entrada e o último nó é um nó de saída do grafo G .
- ✓ Seja $IN(x)$ e $OUT(x)$ o número de arcos que entram e que saem do nó x , respectivamente. Assumimos $IN(s) = 0$, tal que s é o nó de entrada, e $OUT(o) = 0$, tal que o é o nó de saída.

Os *Casos de Teste* no **Teste Estrutural** devem:

- ✓ Garantir que todos os *caminhos independentes* de um módulo tenham sido exercitados pelo menos uma vez
- ✓ Exercitem todas as *decisões lógicas* em seus lados verdadeiro e falso
- ✓ Executem todos os *ciclos nos seus limites* e dentro de seus intervalos operacionais
- ✓ Exercitem as *condições*

- ✓ O **Grafo de Fluxo** mostra o fluxo de controle, isto é, o código é modelado como um *control flow graph* (CFG)
- ✓ *Control flow graphs* associam uma borda com cada ramificação possível no programa e um nó com sequências de instruções. Logo:

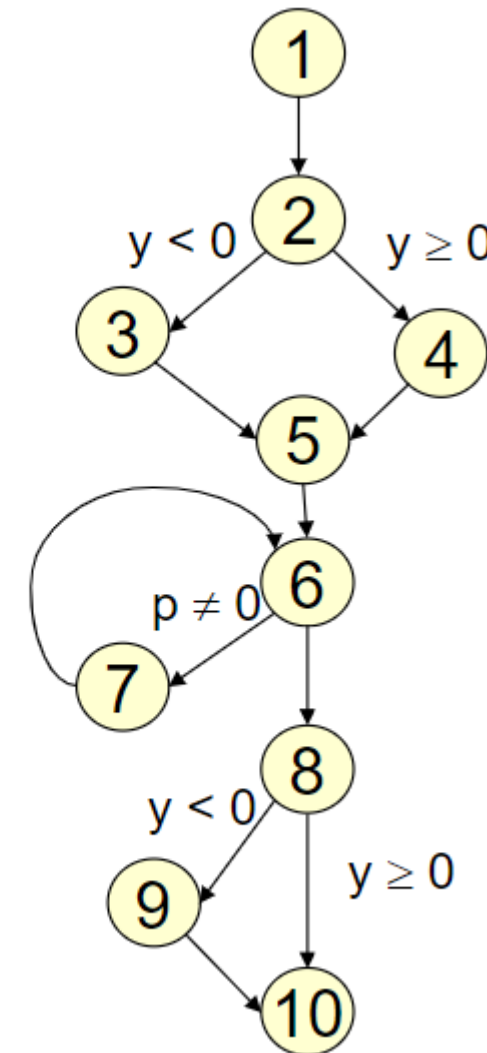


Cálculo de x^y

```

1.  read x, y;
2.  if y < 0
3.    then p := 0 - y
4.    else p := y;
5.  z := 1.0;
6.  while p ≠ 0 do
7.    begin
        z := z * x; p := p - 1;
    end;
8.  if y < 0
9.    then z := 1 / z;
10. write z;
    end;

```



✓ Cobertura de Nodo (*Node coverage*)

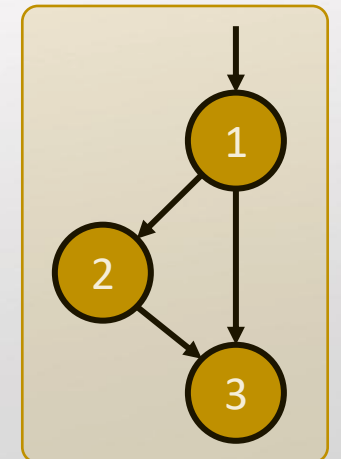
- Analisa se os teus testes cobriram **todos os nodos** do teu programa.
- Requisito de teste: {1,2,3}
- Caminho do teste: [1,2,3]

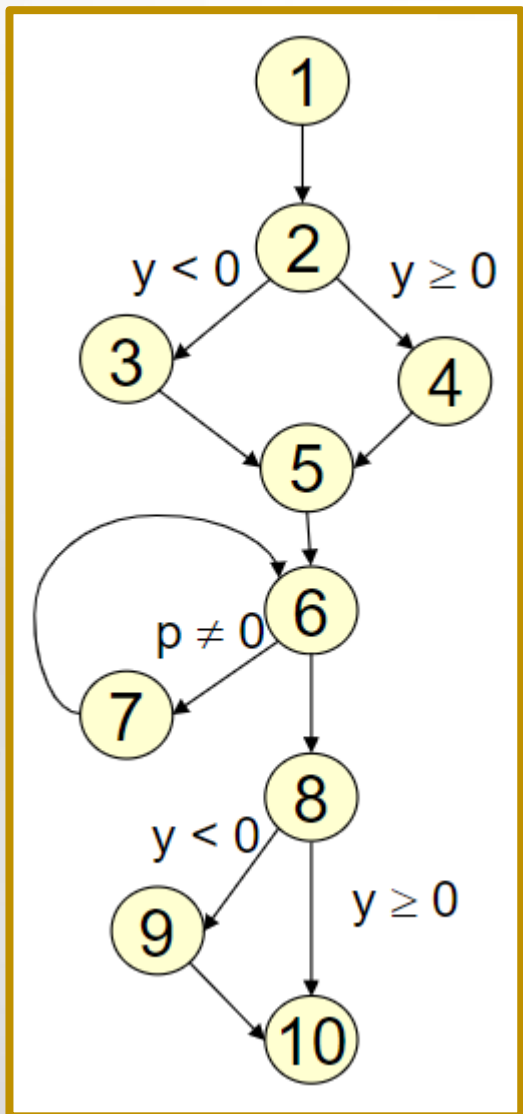
✓ Cobertura de Aresta (*Edge coverage*)

- Analisa se os teus testes cobriram **todas as arestas** do teu programa.
- Requisito de teste: {(1,2), (1,3), (2,3)}
- Caminho do teste: [1,2,3], [1,3]

✓ Cobertura de Caminho (*Complete Path Coverage*)

- Analisa se os teus testes cobriram **todos os caminhos** do teu programa.
- Caminho do teste: [1,2,3], [1,3]

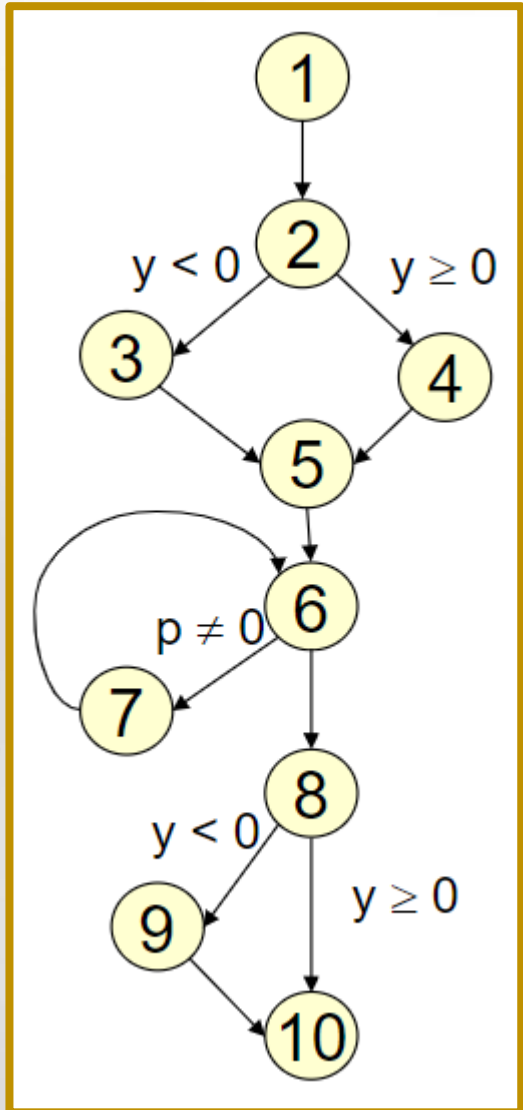




✓ Cobertura de Nodo (*Node coverage*)

- Analisa se os teus testes cobriram **todos os nodos** do teu programa.

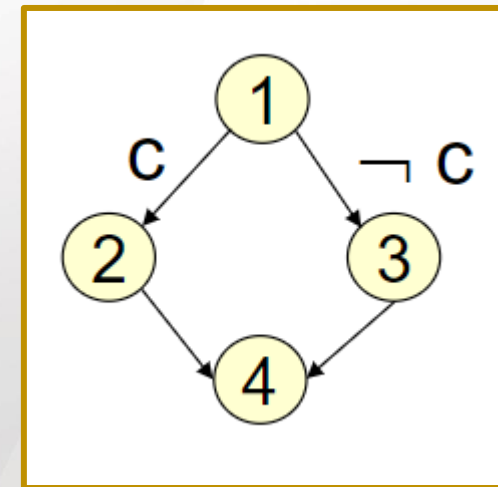
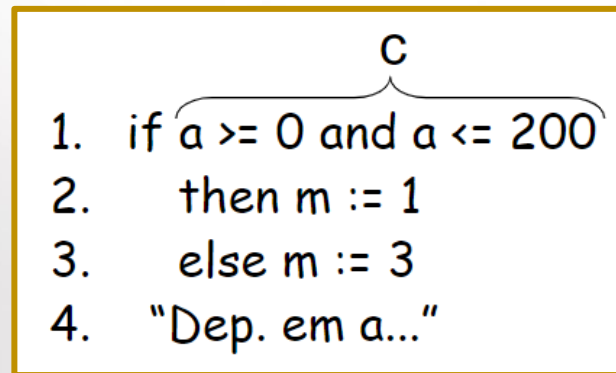
nós	predicados	dados
{1,2,3,5,6,7,6, 8,9,10}	$\forall x, y < 0$	(4, -1)
{1,2,4,5,6,8,10}	$\forall x, y = 0$	(4, 0)



- ✓ Cobertura de Caminho/Ramos (*Complete Path Coverage*)
 - Analisa se os teus testes cobriram **todos os caminhos** do teu programa.
- ✓ Cobertura de Aresta (*Edge coverage*)
 - Analisa se os teus testes cobriram **todas as arestas** do teu programa.

ramos	predicados	dados
{(1,2), (2,3), (3,5), (5,6), (6,7), (7,6), (6,8), (8,9), (9,10)}	$\forall x, y < 0$	(4, -1)
{(1,2), (2,4), (4,5), (5,6), (6,7), (7,6), (6,8), (8, 10)}	$\forall x, y = 0$	(4, 0)

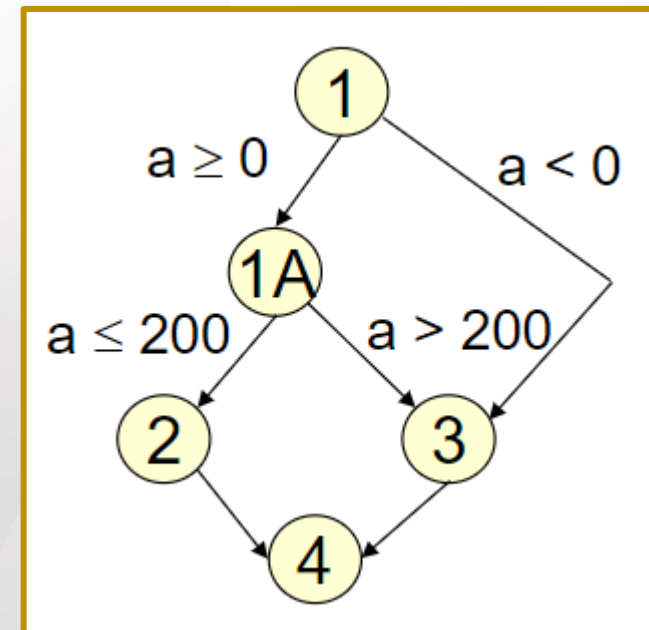
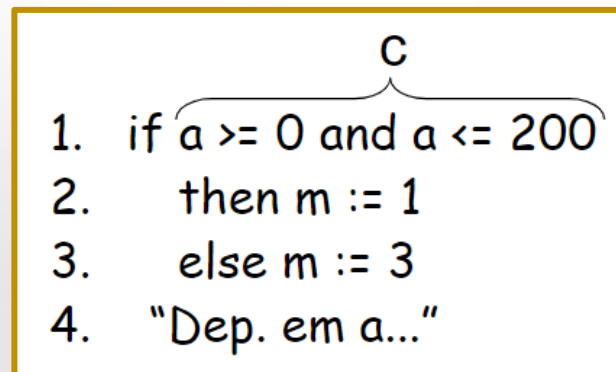
- ✓ Cobertura de Condição (*Condition coverage / Predicate coverage*)
 - É a cobertura na qual cada uma das **expressões booleanas** foi avaliada como *verdadeira* e *falsa*.



ramos	dados
$\{(1,2), (2,4)\}$	$a = 5$
$\{(1,3), (3,4)\}$	$a = -5$

✓ Cobertura de Condição (*Condition coverage / Predicate coverage*)

- É a cobertura na qual cada uma das **expressões booleanas** foi avaliada como *verdadeira* e *falsa*.



ramos	dados
{(1,1A), (1A, 2), (2,4)}	a = 5
{(1,1A), (1A,3), (3,4)}	a = 500
{(1,3), (3,4)}	a = -5

Todas as condições devem ser avaliadas para valores V/F em cada (sub)decisão

EXERCÍCIO

- ✓ Dado o exemplo do software *Identifier*, do livro do Maldonado:
 - Fazer o CFG do código
 - Determine os testes para obter 100% de *cobertura de nodo*, de *cobertura de aresta* e de *cobertura de caminho*


```
#include <stdio.h>
main ()
{
    char  achar;
    int   length, valid_id;
    length = 0;
1   printf ("Digite um possível identificador\n");
    printf ("seguido por <ENTER>: ");
    achar = fgetc (stdin);
    valid_id = valid_starter (achar);
    if (valid_id)
    {
2       length = 1;
    }
3   achar = fgetc (stdin);
4   while (achar != '\n')
5   {
6       if (!(valid_follower (achar)))
7       {
8           valid_id = 0;
9       }
10      length++;
      achar = fgetc (stdin);
8   if (valid_id && (length >= 1) &&
    (length < 6) )
9   {
10      printf ("Valido\n");
    }
    else
10  {
11      printf ("Invalido\n");
    }
11 }
```

```
int valid_starter (ch)
char ch;
{
```

```
1   if (((ch >= 'A') &&
      (ch <= 'Z')) ||
      ((ch >= 'a') &&
      (ch <= 'z')))
```

```
2       return (1);
```

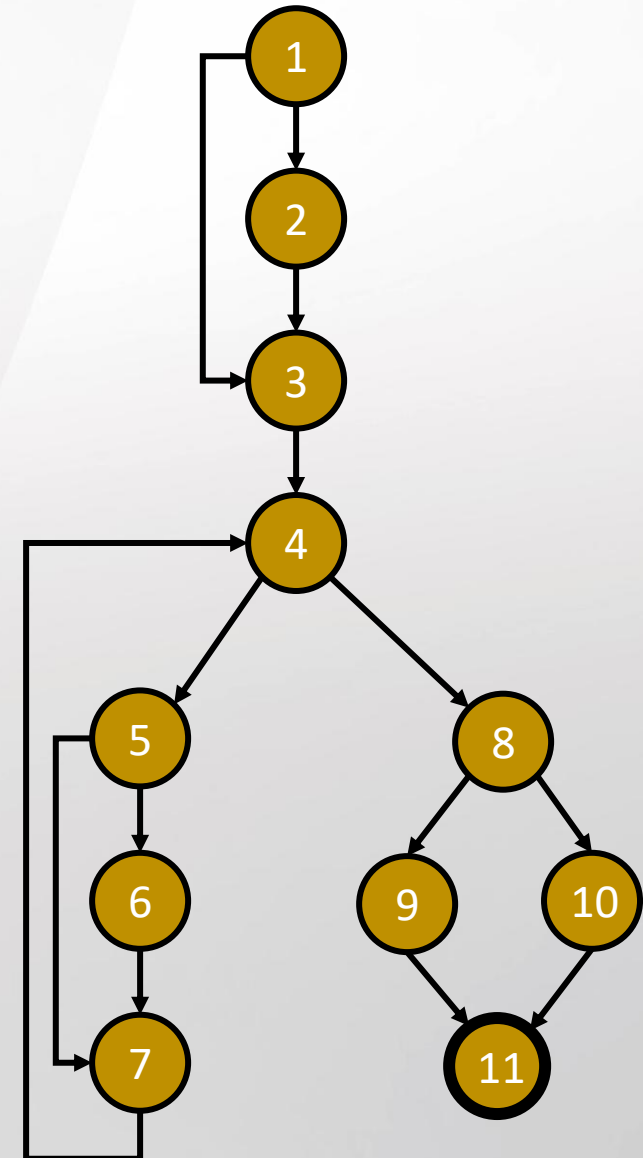
```
3   else
      return (0);
}
```

```
int valid_follower (ch)
char ch;
{
```

```
1   if (((ch >= 'A') &&
      (ch <= 'Z')) ||
      ((ch >= 'a') &&
      (ch <= 'z')) ||
      ((ch >= '0') &&
      (ch <= '9')))
```

```
2       return (1);
```

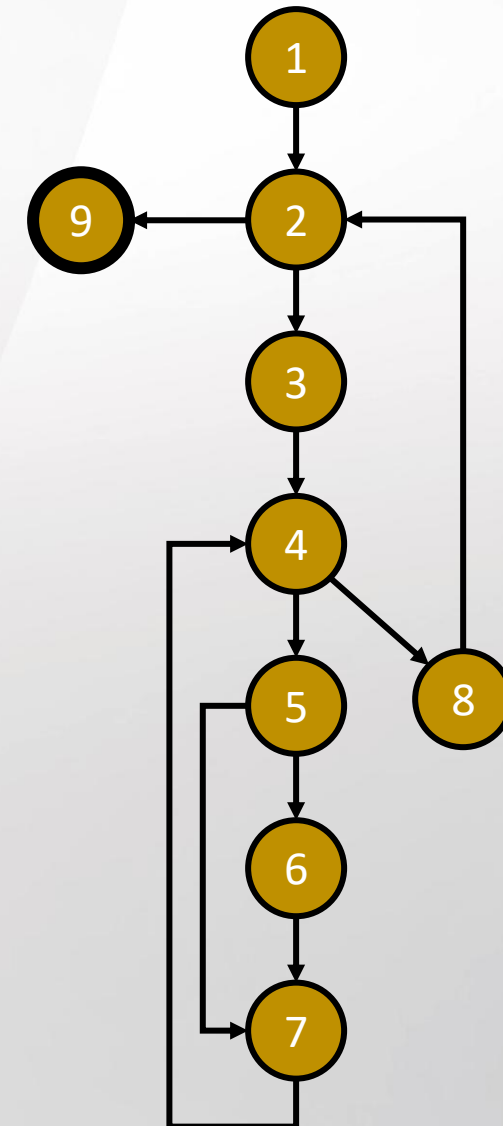
```
3   else
      return (0);
}
```



```

void sort(int *v, int size){
1   int i = 0;
2   while (i < size-1) {
3       int j = 0;
4       while (j < size-1) {
5           if (v[j] > v[j+1]){
6               int aux = v[j];
6               v[j] = v[j+1];
6               v[j+1] = aux;
6               }
7           j = j++;
8       }
8       i = i++;
9  }

```



Limitações:

- ✓ Quando certa funcionalidade não é implementada, nenhum caso de teste será necessário para exercitá-la!

O GCC tem uma ferramenta para gerar o CFG para nós:

`gcc -O0 -g -fdump-tree-cfg-graph -c <arquivo.c>`

A ferramenta dot transforma o .dot gerado em uma imagem:

`dot -Tpng <arquivo.dot> -o cfg.png`

